

Problema B

ChatMunarinho

Arquivo fonte: chatmunarinho.{ c | cpp | java | py }

Autor: Prof. Dr. Leandro Luque (Fatec Mogi das Cruzes/U-TAD Madrid)

Munarinho gosta muito de História, Política e afins. Como seu amigo Jeff não está mais compartilhando dos mesmos gostos, ele vem conversando com LLMs sobre o assunto. Porém, no mês passado, a conta de Munarinho veio muito alta e ele finalmente decidiu implementar sua própria LLM, o ChatMunarinho.

Atualmente, ele está na fase de tokenização — processo de dividir a entrada em grupos de caracteres para depois convertê-los em números (o que um computador realmente entende).

Determinando que bibliotecas prontas são muito fáceis, ele decide implementar sua própria versão do algoritmo *Byte Pair Encoding* (BPE). Porém, como bom inventor de problemas elegantes, ele impõe uma modificação curiosa:

- Em vez de pares, o algoritmo deve considerar **trios consecutivos de tokens**.

Agora, Munarinho quer simular esse processo e entender como sua tokenização evolui ao longo do tempo.

Definição do algoritmo

O processo funciona assim:

1. Inicialize o vocabulário com todos os caracteres distintos da string de entrada (incluindo espaços, vírgulas e pontos).
2. Separe a string em uma lista de tokens (inicialmente, caracteres individuais).
3. Enquanto o tamanho do vocabulário for menor que o limite máximo:
 - (a) Conte todas as ocorrências de **trios consecutivos** de tokens.
 - (b) Escolha o **trio mais frequente**.
 - i. Se nenhum trio aparecer mais de uma vez, interrompa — exceto se restar apenas um trio possível. Neste caso, continue.
 - ii. Em caso de empate, escolha o trio que aparece primeiro se ordenado alfabeticamente.
 - (c) Crie um novo token com o trio.
 - (d) Substitua todas as ocorrências desse trio na sequência atual (da esquerda para a direita, sem sobreposição).
 - (e) Adicione o novo token ao vocabulário.

Exemplo ilustrativo (passo-a-passo)

Considere a string:

abababa

E limite de vocabulário $V = 10$.

Estado inicial:

- Tokens: a b a b a b a
- Vocabulário: a, b

Iteração 1

Trios e frequências:

- (a, b, a): 3
- (b, a, b): 2

Escolhido: (a, b, a)

Novo token: aba

Nova sequência:

aba b aba

Vocabulário: a, b, aba

Iteração 2

Trios:

- (aba, b, aba): 1

Escolhido: (aba, b, aba)

Novo token: abababa

Nova sequência:

abababa

Vocabulário: a, b, aba, abababa

Iteração 3

Não há mais trios possíveis.

Resultado final:

abababa

Entrada

A entrada contém:

- Uma string S ($1 \leq |S| \leq 10^5$), composta potencialmente por caracteres minúsculos, espaços, vírgulas e pontos.
- Um inteiro V ($1 \leq V \leq 1000$), indicando o tamanho máximo do vocabulário.

Saída

Imprima a entrada conforme formatada após o processo de tokenização.

Observações

- A substituição dos trios deve respeitar a ordem da esquerda para a direita, sem sobreposição.
- Após cada adição ao vocabulário, os trios devem ser recalculados com base na nova sequência.
- É garantido que o algoritmo termina.
- Atenção à eficiência: soluções ingênuas podem não passar no tempo limite.

Exemplo de Entrada 1

```
abcabcabc  
1
```

Exemplo de Saída 1

```
a b c a b c a b c
```

Exemplo de Entrada 2

```
abcabcabc  
4
```

Exemplo de Saída 2

```
abc abc abc
```

Exemplo de Entrada 3

abcabcabc
5

Exemplo de Saída 3

abcabcabc

Exemplo de Entrada 4

abcabcabc
6

Exemplo de Saída 4

abcabcabc